15 Minutes Tutorial - Extended

After you have developed your first own DSL, the question arises how the behavior and the semantics of the language can be customized. Here you find a few mini-tutorials that illustrate common use cases when crafting your own DSL. These lessons are independent from each other. Each of them will be based on the language that was built in the previous domainmodel tutorial (102_domainmodelwalkthrough.html).

Writing a Code Generator With Xtend

As soon as you generate the Xtext artifacts from the grammar, a code generator stub is put into the runtime project of your language. Let's dive into Xtend (https://www.eclipse.dev/Xtext/xtend/) and see how you can integrate your own code generator with Eclipse.

In this lesson you will generate Java Beans for entities that are defined in the domainmodel DSL. For each *Entity*, a Java class is generated and each *Feature* will lead to a private field in that class including public getters and setters. For the sake of simplicity, we will use fully qualified names all over the generated code.

```
1. package my.company.blog;
2.
3. public class HasAuthor {
4.     private java.lang.String author;
5.
6.     public java.lang.String getAuthor() {
7.         return author;
8.     }
9.
10.     public void setAuthor(java.lang.String author) {
11.         this.author = author;
12.     }
13. }
```

First of all, locate the file *DomainmodelGenerator.xtend* in the package *org.example.domainmodel.generator*. This Xtend class is used to generate code for your models in the standalone scenario and in the interactive Eclipse environment. Let's make the implementation more meaningful and start writing the code generator. The strategy is to find all entities within a resource and trigger code generation for each one.

1. First of all, you will have to filter the contents of the resource down to the defined entities. Therefore we need to iterate a resource with all its deeply nested elements. This can be achieved with the method `getAllContents()`. To use the resulting TreeIterator (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.common/src/org/eclipse/emf/common/util/TreeIterator.java) in a `for` loop, we use the extension method `toIterable()` from the built-in library class IteratorExtensions (https://github.com/eclipse-

xtext/xtext/blob/main/org.eclipse.xtext.xbase.lib/src/org/eclipse/xtext/xbase/lib/IteratorExtensions.java).

```
1. class DomainmodelGenerator extends AbstractGenerator {
2.
3.     override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGenera
   torContext context) {
4.         for (e : resource.allContents.toIterable.filter(Entity)) {
5.
6.         }
7.     }
8. }
```

2. Now let's answer the question how we determine the file name of the Java class that each *Entity* should yield. This information should be derived from the qualified name of the *Entity* since Java enforces this pattern. The qualified name itself has to be obtained from a special service that is available for each language. Fortunately, Xtend allows to reuse that one easily. We simply inject the IQualifiedNameProvider (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/IQualifiedNameProvider.java) as a field into the generator.

   ```
   1.   @Inject extension IQualifiedNameProvider
   ```

   This allows to ask for the name of an entity. It is straightforward to convert the name into a file name:

   ```
   1. override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorC
      ontext context) {
   2.     for (e : resource.allContents.toIterable.filter(Entity)) {
   3.         fsa.generateFile(
   4.             e.fullyQualifiedName.toString("/") + ".java",
   5.             e.compile)
   6.     }
   7. }
   ```

3. The next step is to write the actual template code for an entity. For now, the function `Entity.compile` does not exist, but it is easy to create it:

   ```
   1. private def compile(Entity e) '''
   2.     package «e.eContainer.fullyQualifiedName»;
   3.
   4.     public class «e.name» {
   5.     }
   6. '''
   ```

4. This small template is basically the first shot at a Java-Beans generator. However, it is currently rather incomplete and will fail if the *Entity* is not contained in a package. A small modification fixes this. The `package` declaration has to be wrapped in an `IF` expression:

```
1. private def compile(Entity e) '''
2.     «IF e.eContainer.fullyQualifiedName !== null»
3.         package «e.eContainer.fullyQualifiedName»;
4.     «ENDIF»
5.
6.     public class «e.name» {
7.     }
8. '''
```

Let's handle the *superType* of an *Entity* gracefully, too, by using another `IF` expression:

```
1. private def compile(Entity e) '''
2.     «IF e.eContainer.fullyQualifiedName !== null»
3.         package «e.eContainer.fullyQualifiedName»;
4.     «ENDIF»
5.
6.     public class «e.name» «IF e.superType !== null
7.             »extends «e.superType.fullyQualifiedName» «ENDIF»{
8.     }
9. '''
```

5. Even though the template will compile the *Entities* without any complaints, it still lacks support for the Java properties that each of the declared features should yield. For that purpose, you have to create another Xtend function that compiles a single feature to the respective Java code.

```
1. private def compile(Feature f) '''
2.     private «f.type.fullyQualifiedName» «f.name»;
3.
4.     public «f.type.fullyQualifiedName» get«f.name.toFirstUpper»() {
5.         return «f.name»;
6.     }
7.
8.     public void set«f.name.toFirstUpper»(«f.type.fullyQualifiedName» «f.name»)
   {
9.         this.«f.name» = «f.name»;
10.    }
11. '''
```

As you can see, there is nothing fancy about this one. Last but not least, we have to make sure that the function is actually used.

```
1. private def compile(Entity e) '''
2.     «IF e.eContainer.fullyQualifiedName !== null»
3.         package «e.eContainer.fullyQualifiedName»;
4.     «ENDIF»
5.
6.     public class «e.name» «IF e.superType !== null
7.             »extends «e.superType.fullyQualifiedName» «ENDIF»{
8.         «FOR f : e.features»
9.             «f.compile»
10.        «ENDFOR»
11.    }
12. '''
```

The final code generator is listed below. Now you can give it a try! Launch a new Eclipse Application (*Run As → Eclipse Application* on the Xtext project) and create a *dmodel* file in a Java Project. Eclipse will ask you to turn the Java project into an Xtext project then. Simply agree and create a new source folder *src-gen* in that project. Then you can see how the compiler will pick up your sample *Entities* and generate Java code for them.

```
package org.example.domainmodel.generator

import com.google.inject.Inject
import org.eclipse.emf.ecore.resource.Resource
import org.eclipse.xtext.generator.AbstractGenerator
import org.eclipse.xtext.generator.IFileSystemAccess2
import org.eclipse.xtext.generator.IGeneratorContext
import org.eclipse.xtext.naming.IQualifiedNameProvider
import org.example.domainmodel.domainmodel.Entity
import org.example.domainmodel.domainmodel.Feature

class DomainmodelGenerator extends AbstractGenerator {

    @Inject extension IQualifiedNameProvider

    override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorContext context) {
        for (e : resource.allContents.toIterable.filter(Entity)) {
            fsa.generateFile(
                e.fullyQualifiedName.toString("/") + ".java",
                e.compile)
        }
    }

    private def compile(Entity e) '''
        «IF e.eContainer.fullyQualifiedName !== null»
            package «e.eContainer.fullyQualifiedName»;
        «ENDIF»

        public class «e.name» «IF e.superType !== null
                »extends «e.superType.fullyQualifiedName» «ENDIF»{
            «FOR f : e.features»
                «f.compile»
            «ENDFOR»
        }
    '''

    private def compile(Feature f) '''
        private «f.type.fullyQualifiedName» «f.name»;

        public «f.type.fullyQualifiedName» get«f.name.toFirstUpper»() {
            return «f.name»;
        }

        public void set«f.name.toFirstUpper»(«f.type.fullyQualifiedName» «f.name») {
            this.«f.name» = «f.name»;
        }
    '''
}
```

If you want to play around with Xtend, you can try to use the Xtend tutorial which can be materialized into your workspace. Simply choose *New → Example → Xtend Examples → Xtend Introductory Examples* and have a look at the features of Xtend. As a small exercise, you could

implement support for the *many* attribute of a *Feature* or enforce naming conventions, e.g. generated field names should start with an underscore.

Creating Custom Validation Rules

One of the main advantages of DSLs is the possibility to statically validate domain-specific constraints. Because this is a common use case, Xtext provides a dedicated hook for this kind of validation rules. In this lesson, we want to ensure that the name of an *Entity* starts with an upper-case letter and that all features have distinct names across the inheritance relationship of an *Entity*.

Locate the class *DomainmodelValidator* in the package *org.example.domainmodel.validation* of the language project. Defining the constraint itself is only a matter of a few lines of code:

```
1. public static final String INVALID_NAME = "invalidName";
2.
3. @Check
4. public void checkNameStartsWithCapital(Entity entity) {
5.     if (!Character.isUpperCase(entity.getName().charAt(0))) {
6.         warning("Name should start with a capital",
7.             DomainmodelPackage.Literals.TYPE__NAME,
8.             INVALID_NAME);
9.     }
10. }
```

Any name for the method will do. The important thing is the Check (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/Check.java) annotation that advises the Xtext framework to use the method as a validation rule. If the name starts with a lower case letter, a warning will be attached to the name of the *Entity*.

The second validation rule is straight-forward, too. We traverse the inheritance hierarchy of the *Entity* and look for features with equal names.

```
1. @Check
2. public void checkFeatureNameIsUnique(Feature feature) {
3.     Entity superEntity = ((Entity) feature.eContainer()).getSuperType();
4.     while (superEntity != null) {
5.         for (Feature other : superEntity.getFeatures()) {
6.             if (Objects.equal(feature.getName(), other.getName())) {
7.                 error("Feature names have to be unique", DomainmodelPackage.Litera
   ls.FEATURE__NAME);
8.                 return;
9.             }
10.         }
11.         superEntity = superEntity.getSuperType();
12.     }
13. }
```

The sibling features that are defined in the same entity are automatically validated by the Xtext framework, thus they do not have to be checked twice. Note that this implementation is not optimal in terms of execution time because the iteration over the features is done for all features of each entity.

You can determine when the `@Check` -annotated methods will be executed with the help of the CheckType (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/CheckType.java) enum. The default value is `FAST` , i.e. the checks will be executed on editing, saving/building or on request; also available are `NORMAL` (executed on build/save or on request) and `EXPENSIVE` (executed only on request).

```
@Check(CheckType.NORMAL)
public void checkFeatureNameIsUnique(Feature feature) {
    ...
}
```

Unit Testing the Language

Automated tests are crucial for the maintainability and the quality of a software product. That is why it is strongly recommended to write unit tests for your language. The Xtext project wizard creates two test projects for that purpose. These simplify the setup procedure for testing the basic language features and the Eclipse UI integration.

This tutorial is about testing the parser, the linker, the validator and the generator of the *Domainmodel* language. It leverages Xtend to write the test cases.

1. The core of the test infrastructure is the XtextRunner (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/XtextRunner.java) (for JUnit 4) and the language-specific IInjectorProvider (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/IInjectorProvider.java). Both have to be provided by means of class annotations. An example test class should have already been generated by the Xtext code generator, named *org.example.domainmodel.tests.DomainmodelParsingTest*:

```
@RunWith(XtextRunner)
@InjectWith(DomainmodelInjectorProvider)
class DomainmodelParsingTest {

    @Inject
    ParseHelper<Domainmodel> parseHelper

    @Test
    def void loadModel() {
        val result = parseHelper.parse('''
            Hello Xtext!
        ''')
        Assert.assertNotNull(result)
        val errors = result.eResource.errors
        Assert.assertTrue('''Unexpected errors: «errors.join(", ")»''', errors.isEmpty)
    }

}
```

Note: When using JUnit 6 the InjectionExtension (https://github.com/eclipse-xtext/xtext/blob/

main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/extensions/InjectionExtension.java) is used instead of the XtextRunner. The Xtext code generator generates the example slightly different, depending on which option you have chosen in the *New Xtext Project* wizard.

2. The utility class ParseHelper (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/util/ParseHelper.java) allows to parse an arbitrary string into a *Domainmodel*. The model itself can be traversed and checked afterwards. A static import of Assert (https://junit.org/junit4/javadoc/4.13/org/junit/Assert.html) leads to concise and readable test cases. You can rewrite the generated test case as follows:

```
import static org.junit.Assert.*

...

    @Test
    def void parseDomainmodel() {
        val model = parseHelper.parse(
            "entity MyEntity {
                parent: MyEntity
            }")
        val entity = model.elements.head as Entity
        assertSame(entity, entity.features.head.type)
    }
```

3. In addition, the utility class ValidationTestHelper (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/validation/ValidationTestHelper.java) allows to test the custom validation rules written for the language. Both valid and invalid models can be tested.

```
@Inject ParseHelper<Domainmodel> parseHelper

@Inject ValidationTestHelper validationTestHelper

@Test
def testValidModel() {
    val entity = parseHelper.parse(
        "entity MyEntity {
            parent: MyEntity
        }")
    validationTestHelper.assertNoIssues(entity)
}

@Test
def testNameStartsWithCapitalWarning() {
    val entity = parseHelper.parse(
        "entity myEntity {
            parent: myEntity
        }")
    validationTestHelper.assertWarning(entity,
        DomainmodelPackage.Literals.ENTITY,
        DomainmodelValidator.INVALID_NAME,
        "Name should start with a capital"
    )
}
```

You can further simplify the code by injecting `ParseHelper` and `ValidationTestHelper` as extensions. This feature of Xtend allows to add new methods to a given type without modifying it. You can read more about extension methods in the Xtend documentation (https://www.eclipse.dev/Xtext/xtend/documentation/202_xtend_classes_members.html#extension-methods). You can rewrite the code as follows:

```
@Inject extension ParseHelper<Domainmodel>

@Inject extension ValidationTestHelper

@Test
def testValidModel() {
    "entity MyEntity {
    parent: MyEntity
    }".parse.assertNoIssues
}

@Test
def testNameStartsWithCapitalWarning() {
    "entity myEntity {
        parent: myEntity
    }".parse.assertWarning(
        DomainmodelPackage.Literals.ENTITY,
        DomainmodelValidator.INVALID_NAME,
        "Name should start with a capital"
    )
}
```

4. The CompilationTestHelper (https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.xbase.testing/src/org/eclipse/xtext/xbase/testing/CompilationTestHelper.java) utility class comes in handy while unit testing the custom generators:

```
@Inject extension CompilationTestHelper

@Test def test() {
    '''
        datatype String

        package my.company.blog {
            entity Blog {
                title: String
            }
        }
    '''.assertCompilesTo('''
        package my.company.blog;

        public class Blog {
            private String title;

            public String getTitle() {
                return title;
            }

            public void setTitle(String title) {
                this.title = title;
            }
        }
    ''')
    }
```

5. After saving the Xtend file, it is time to run the tests. Select *Run As → JUnit Test* from the editor's context menu. All implemented test cases should succeed.

These tests serve only as a starting point and can be extended to cover the different features of the language. As a small exercise, you could implement e.g. test cases for the `checkFeatureNameIsUnique` validation rule. You can find more test cases in the example projects shipped with the Xtext Framework. Simply go to *File → New → Example → Xtext Examples* to instantiate them into your workspace.

Next Chapter: Five simple steps to your JVM language (104_jvmdomainmodel.html)